



N0JCG
OPEN RADIO PLATFORM

OPERATOR HANDBOOK

N0JCG Scanner

Installation, radio setup, operation, and troubleshooting

A complete guide to the split-host P25, VHF, and UHF scanning system, browser audio, radio profiles, and stable RTL-SDR serial assignments.

RELEASE

4.1.0

RADIO PATHS

P25 / VHF / UHF

PUBLICATION

August 2026

AUDIENCE

Operators and maintainers

Contents

Use this guide from initial hardware setup through daily operation and maintenance.

1. Getting started: new installation
2. What N0JCG Scanner does
3. Safety and legal use
4. Hardware and network requirements
5. RTL-SDR role and serial-number plan
6. Prepare the Raspberry Pi
7. Program RTL-SDR serial numbers
8. Apply and verify receiver assignments
9. Application files and services
10. Start N0JCG Scanner and open the web application
11. Use the Dashboard
12. Use Skip, Block, Clear lock, and Clear blocks
13. Manage radio profiles
14. Import and export analog CHIRP CSV files
15. Import and export P25 CSV files
16. Understand the audio arbitrator
17. Routine maintenance and backups
18. Troubleshooting
19. Technical reference
20. Registration and five-minute trial
21. Acceptance checklist

Receiver ownership is serial-first. Use the private station role map for VHF and UHF. Linux device indexes are never permanent role assignments.

This manual covers the N0JCG Scanner v4.2.0 production layout: P25 trunked radio, FFT-directed VHF and UHF analog scanning, unified browser audio, radio profiles, and four dedicated RTL-SDR receiver assignments. It is written for both initial installation and normal daily operation.

Getting started: new installation

This section is for a new scanner appliance that has not yet been configured. The complete first-install wizard is checked into the repository, so the operator does not need to copy individual service files or manually assemble the receiver configuration.

Hardware you need

- Raspberry Pi 5 (4 GB or more recommended) with a reliable USB-C power supply.
- A high-endurance microSD card, 32 GB or larger, for Raspberry Pi OS and the scanner software. A 64 GB card is preferred when retaining diagnostic logs.
- A powered USB 3 hub with enough ports and current capacity for four RTL-SDR receivers. Avoid unpowered hubs for a permanent installation.

- Four RTL-SDR dongles with direct USB access. The installer assigns stable serial numbers to each one; do not connect all four during serial setup.
- A network connection from the Pi to the operator browser. Wired Ethernet is preferred; Wi-Fi is acceptable where Ethernet is unavailable.
- Receive-only antennas, coax, and any needed band filters or splitters.

Antenna recommendations

Use antennas appropriate to the bands you intend to monitor. A 2 m/VHF scanner antenna is appropriate for the VHF FFT receiver, and a 70 cm/UHF antenna is appropriate for the UHF receiver. P25 700/800 MHz systems benefit from a dedicated 700/800 MHz antenna; a directional Yagi can improve a known site, while a discone is useful for broad coverage. Keep antennas separated from transmit antennas and use filtering or attenuation if a nearby transmitter overloads an SDR. The P25 control and voice receivers may share an antenna through a suitable splitter, but each SDR must remain a separate USB receiver.

Prepare the Pi and SD card

1. Use Raspberry Pi Imager on a workstation to write a current 64-bit Raspberry Pi OS image to the microSD card.
2. In the Imager customization panel, set the hostname, enable SSH, create the intended Linux user, and configure the network when practical.
3. Boot the Pi with the display, keyboard, network, and powered USB hub attached. Complete the first-boot prompts and apply OS updates.
4. Confirm the Pi's IP address and verify SSH from the MSYS/Git Bash machine:

```
ssh <user>@<pi-ip>
```

Do not attach the RTL-SDRs until the installer reaches the serial-assignment prompts. This prevents Linux device indexes from being confused during the one-at-a-time assignment process.

Run the first-install wizard

From the checked-out repository on the workstation, in MSYS/Git Bash, run:

```
cd /c/msys64/home/<windows-user>/sdrdev/PI-SCANNER
./tools/install_n0jcg_scanner_pi.sh
```

The wizard asks for the Pi IP address, SSH user (default `pi`), and SSH password. It saves reusable values in the local `.env` file. That file contains a password and must remain private.

The wizard installs the required OS packages, deploys the application to `/home/<user>/n0jcg-scanner`, installs path-correct systemd units, and leaves all scanner services stopped. It then prompts for each receiver in turn:

1. Insert only the P25 Control SDR; the wizard assigns the required production serial, verifies it, and asks you to remove it.
2. Insert only the P25 Voice SDR; it assigns and verifies its required production serial.
3. Insert only the VHF / 2 m SDR; it assigns and verifies its required production serial.
4. Insert only the UHF / 70 cm SDR; it assigns and verifies its required production serial.

The serial numbers are required production assignments and are not entered manually. If an assignment fails, stop and correct the physical USB connection before continuing. The wizard validates the RTL tools, Python modules, role map, and service files before it finishes. It does not start the services.

Load the first radio profile

Open `http://<pi-ip>:8070/` after installation. Because no named profile exists on a new installation, the application opens directly on **Radio setup**. Use the downloadable P25 and CHIRP CSV templates to create/import a profile, review the receiver roles and frequencies, and save the profile. Only after the role map and profile validate should the administrator enable and start the scanner services using the commands printed by the wizard.

1. What N0JCG Scanner does

N0JCG Scanner combines three scanning paths, using four dedicated RTL-SDR receivers, in one touch-friendly web application:

- **P25:** follows permitted, clear P25 talkgroups using dedicated control and voice RTL-SDR receivers. The production P25 assignment is configured through the role template; the launch path fails closed rather than silently reverting to a single-radio decoder.
- **VHF:** surveys only uploaded VHF channels with an FFT, validates an active carrier, demodulates NFM audio, and returns to scanning when the call ends.
- **UHF:** uses the same FFT-directed workflow for uploaded UHF channels.
- **Audio arbitrator:** accepts P25, VHF, and UHF audio and sends one active source to the browser without mixing calls together.

Encrypted P25 audio is not decoded. Encrypted or blocked traffic is detected and muted or skipped.

2. Safety and legal use

- Monitor only traffic that may lawfully be received in your location.
- Do not attempt to decrypt encrypted traffic or recover encryption keys.
- N0JCG Scanner is receive-only. Never connect a transmitter directly to an RTL-SDR input.
- A nearby transmitter can overload or damage an SDR. Use suitable antenna separation, filtering, and attenuation during transmitter tests.
- Stop the service that owns a receiver before running direct `rtl_test`, `rtl_eeprom`, `rtl_tcp`, or `rtl_fm` commands against it.

3. Hardware and network requirements

The validated production system uses:

- Raspberry Pi 5 running 64-bit Debian 13 / Raspberry Pi OS Trixie.
- Reliable Pi 5 power supply.
- Four uniquely serialized RTL-SDR receivers dedicated to N0JCG Scanner.
- A powered USB hub suitable for the combined current draw of the receivers.
- Appropriate antennas or a properly engineered receive-only distribution system.
- Wired Ethernet or reliable Wi-Fi on the same network as the operator device.
- A modern browser with Web Audio support.

For best RF performance, label every dongle physically after assigning its serial. Keep antenna leads and USB cables identifiable. VHF, UHF, and 700/800 MHz P25 benefit from band-appropriate antennas and filters.

4. RTL-SDR role and serial-number plan

Linux device indexes such as `device 0` change after reboots and USB reconnection. N0JCG Scanner therefore owns receivers by the stable eight-digit serial stored in each RTL-SDR EEPROM.

Role	Required serial	Operational use
P25 control	<P25_CONTROL_SERIAL>	Remains on the trunked-system control channel
P25 voice	<P25_VOICE_SERIAL>	Follows P25 voice-channel grants
VHF / analog 2 m	<VHF_SERIAL>	FFT-directed VHF NFM scanner
UHF / analog 70 cm	<UHF_SERIAL>	FFT-directed UHF NFM scanner

Keep the two analog assignments distinct and do not swap them. The VHF and UHF workers fail closed if their runtime serial or audio port is wrong.

The P25 path supports a fixed-center wideband voice receiver. When enabled, OP25 selects configured voice channels digitally within the sampled bandwidth without a slow RTL hardware retune for every grant. Keep system frequencies, sample rates, demodulator types, and gain values in the ignored station runtime configuration.

5. Prepare the Raspberry Pi

5.1 Install baseline packages

On the Pi:

```
sudo apt update
sudo apt install -y git rtl-sdr sox netcat-openbsd usbutils python3 python3-numpy
```

The P25 receiver also requires the validated OP25 installation. The deployed system uses:

```
/home/pi/op25/op25/gr-op25_repeater/apps/rx.py
```

Use the guarded project probes and installer workflow rather than guessing an OP25 command:

```
cd /home/pi/n0jcg-scanner
./tools/pi5_p25_op25_install_probe.sh
./tools/pi5_p25_op25_postinstall_probe.sh
./tools/pi5_p25_op25_live_command_probe.sh --dry-run
```

5.2 Prevent the television driver from claiming RTL-SDR devices

The DVB kernel modules must be blacklisted for SDR use:

```
printf '%s\n' \
'blacklist dvb_usb_rtl28xxu' \
'blacklist rtl2832' \
'blacklist rtl2830' \
| sudo tee /etc/modprobe.d/rtl-sdr-blacklist.conf
sudo reboot
```

After reboot, this command should normally print no matching modules:

```
lsmod | grep -E 'dvb_usb_rtl28xxu|rtl2832|rtl2830'
```

6. Program RTL-SDR serial numbers

Serial programming is safest with one receiver connected at a time.

6.1 Stop every service that may own an SDR

```
sudo systemctl stop pi-p25-scanner.service
sudo systemctl stop pi-scanner-vhf-worker.service
sudo systemctl stop pi-scanner-uhf-worker.service
```

Also stop any other application that owns an SDR connected to the Pi. Confirm that no unrelated receiver process still owns the device before changing its EEPROM.

6.2 Disconnect all RTL-SDR receivers

Unplug every RTL-SDR. Connect only the receiver that will receive the first serial. Confirm that exactly one receiver is visible:

```
rtl_test -t
```

6.3 Write the serial

For example, to prepare the VHF receiver:

```
sudo rtl_eeprom -d 0 -s <VHF_SERIAL>
```

Read the warning, confirm the write, then unplug and reconnect the receiver. Verify it:

```
rtl_eeprom -d 0
```

Repeat the one-at-a-time procedure for each role in the table above. Write the role and serial on a physical label before moving to the next receiver.

Important rules:

- Use exactly eight numeric digits, including leading zeroes.
- Never give two connected receivers the same serial.
- The `-d 0` index is safe here only because exactly one receiver is connected.
- Never store runtime USB indexes as application role assignments.
- Power-cycle or unplug/reconnect a receiver after changing its EEPROM.

6.4 Verify all receivers together

Reconnect all four N0JCG Scanner receivers while the scanner services remain stopped:

```
rtl_test -t
```

The first lines should list every expected serial exactly once. To inspect each EEPROM individually, use the temporary indexes shown by `rtl_test`:

```
rtl_eeprom -d 0
rtl_eeprom -d 1
```

Continue through the last displayed index. The index order may change later; only the printed serial is persistent.

7. Apply and verify receiver assignments

7.1 Apply the canonical four-receiver map

From the P25 application directory, preview the role map:

```
cd /home/pi/n0jcg-scanner
./tools/pi5_apply_receiver_serial_map.sh --dry-run
```

Apply it after verifying the table:

```
./tools/pi5_apply_receiver_serial_map.sh --apply --yes
```

The tool validates uniqueness, backs up an existing registry, and writes:

```
/home/pi/n0jcg-scanner/runtime/settings/receiver_roles.json
```

7.2 Verify the application inventory

Start the backend, then query the inventory:

```
sudo systemctl restart pi-p25-scanner.service
curl -fsS http://127.0.0.1:8070/api/receivers/inventory \
| python3 -m json.tool
```

Confirm that the four N0JCG Scanner roles have the serials shown above and that no scanner receiver is missing or duplicated. A shared Pi may list receivers owned by other applications; those devices are outside the scope of this manual.

7.3 Verify the analog worker map

The analog runtime configuration is separate from the inventory registry:

```
/home/pi/n0jcg-scanner/runtime/settings/analog_receivers.json
```

Check it through the application API:

```
curl -fsS http://127.0.0.1:8070/api/analog/status \
| python3 -m json.tool
```

Confirm:

- `analog_2m.rtl_serial` matches `<VHF_SERIAL>`.
- `analog_70cm.rtl_serial` matches `<UHF_SERIAL>`.
- Before **Start scanning + audio** is pressed, both workers are expected to be stopped.
- After **Start scanning + audio** is pressed, both roles report `fft_scanning`, `locked`, or another healthy running state.

8. Application files and services

The validated Pi layout uses one application root:

Path	Purpose
<code>/home/pi/n0jcg-scanner</code>	Complete web UI, backend, P25 and analog configuration, profiles, receiver registry, workers, audio arbitrator, controls, and runtime diagnostics

Runtime settings are intentionally not committed to Git. Back them up before replacing an SD card or performing a major upgrade.

Main services:

Service	Purpose
<code>pi-p25-scanner.service</code>	Boot-enabled web UI/API on port 8070 and coordinated scanner control
<code>pi-p25-raw-audio-bridge.service</code>	Boot-enabled three-source audio arbitrator and browser stream on port 8072
<code>pi-p25-audio-pool.service</code>	Boot-enabled collector for P25 voice-receiver audio
<code>pi-scanner-vhf-worker.service</code>	On-demand VHF FFT scanner using <code><VHF_SERIAL></code> ; started and stopped by the dashboard
<code>pi-scanner-uhf-worker.service</code>	On-demand UHF FFT scanner using <code><UHF_SERIAL></code> ; started and stopped by the dashboard

Install or refresh the complete audio/runtime units from the application root:

```
cd /home/pi/n0jcg-scanner
sudo ./tools/install_audio_runtime_units.sh
```

9. Start N0JCG Scanner and open the web application

The radio Pi at `<RADIO_HOST>` runs the complete scanner application: web UI, backend, audio infrastructure, OP25, and analog workers. The N0JCG ROC at `<ROC_HOST>:8095` remains a separate platform dashboard and provides a direct link to the Pi scanner on port `8070`; it does not host or proxy scanner files.

On the radio Pi, enable the radio API and audio infrastructure at boot while keeping the receiver workers out of the boot target:

```
sudo systemctl enable --now pi-p25-scanner.service
sudo systemctl enable --now pi-p25-raw-audio-bridge.service
sudo systemctl enable --now pi-p25-audio-pool.service
sudo systemctl disable --now pi-scanner-vhf-worker.service
sudo systemctl disable --now pi-scanner-uhf-worker.service
```

On the ROC, the existing `n0jcg-roc.service` owns only the dashboard. It does not host or proxy scanner assets.

Open this address from a device on the same network:

```
http://<RADIO_HOST>:8070/
```

The ROC and radio Pi should both have DHCP reservations. The direct Pi URL is the normal scanner operator URL.

For the separate compact phone dashboard, open:

```
http://<RADIO_HOST>:8070/mobile.html
```

Supported phone browsers are redirected to this page automatically when they open the main dashboard address. The **Full dashboard** link bypasses that redirect for the current navigation when radio setup or logs are needed.

The phone dashboard provides the coordinated Start and Stop controls, local Mute and Volume, current P25 or analog activity, Voice/VHF/UHF counters, and analog Skip, Block, Clear lock, and Clear blocks. Use **Full dashboard** when profile editing, logs, or detailed radio setup is required.

After boot, the dashboard and audio infrastructure are available, but P25, VHF, and UHF scanning are all stopped. Opening the web page or the desktop shortcut does not start a receiver. Press **Start scanning + audio** once to start all three scanners together and connect that browser tab to the audio stream. Browsers require a real tap or click before audio can begin.

When another browser has already started the scanners, the new browser shows an enabled **Listen** button. Pressing **Listen** attaches only that browser to the PCM fanout and does not restart P25, VHF, or UHF.

If any one of the three scanners cannot start, the coordinated start is treated as failed and the application returns the other scanners to the stopped state.

10. Use the Dashboard

Top status indicators

- **Scanning:** the system is ready and looking for activity.
- **P25/VHF/UHF ON AIR:** the audio arbitrator is currently forwarding that source.
- **Connected:** the browser can reach the Pi scanner API and audio services.
- **Offline:** check the Pi scanner service and the network path to the radio Pi. The ROC dashboard is not required for direct Pi operation.

Main controls

- **Start scanning + audio:** starts P25, VHF, and UHF scanning together and connects low-latency browser audio.
- **Listen:** appears when the scanners are already running but this browser is not attached; it connects this tab without restarting radio services.
- **Mute / Unmute:** mutes only this browser tab. It does not stop the scanners or mute another browser.
- **Volume:** controls only this browser tab. Moving it while muted also unmutes the tab.

- **Stop:** stops P25, VHF, and UHF scanning together, resets **Voice Calls**, **VHF Locks**, and **UHF Locks** to zero, and disconnects audio in this browser.

After pressing **Stop**, the dashboard and audio services remain online so a later press of **Start scanning + audio** can resume reception without rebooting the Pi.

Activity information

- **Active / Last Talkgroup:** current or most recently heard P25 talkgroup.
- **Voice:** current or last P25 voice frequency.
- **Control:** active P25 control-channel frequency.
- **Phase:** detected P25 phase when known.
- **Audio Arbitrator:** browser connection and active-source status.
- **Active Source:** P25, VHF, UHF, or None.
- **Voice Calls:** count of distinct P25 voice transmissions since the last press of **Stop**.
- **Unique TGIDs:** number of unique P25 talkgroups observed in the current backend session.
- **VHF Locks / UHF Locks:** accepted analog carrier locks; these service counters restart when their worker restarts.
- **Muted:** encrypted or otherwise muted P25 events detected by the parser.

Open **Menu** → **Logs and details** for recent activity, backend/OP25 log tail, active configuration, validated command, and launch-readiness information.

11. Use Skip, Block, Clear lock, and Clear blocks

These buttons become available only while VHF or UHF has a current identified lock.

- **Skip 10 min:** releases the current analog channel and makes it ineligible for 600 seconds.
- **Block channel:** releases the current analog channel and blocks it until blocks are cleared.
- **Clear lock:** releases the current carrier immediately without creating a skip or block.
- **Clear blocks:** removes all temporary skips and persistent analog blocks across both VHF and UHF.

Use **Skip 10 min** for intermittent noise and **Block channel** for a channel that repeatedly opens on interference. Use **Clear lock** when a carrier is stuck but should remain eligible for future calls.

Control state is stored atomically at:

```
/home/pi/n0jcg-scanner/runtime/settings/analog_controls.json
```

12. Manage radio profiles

Open **Menu** → **Radio setup**.

A named profile contains P25 systems/talkgroups and a snapshot of the analog VHF/UHF lists. Receiver serial assignments are hardware settings and are not changed when a profile is loaded.

Save the current setup

1. Enter a name in **New Profile Name**.

2. Press **Save Current**.
3. Press **Refresh Profiles** if the new name does not appear immediately.

Load a profile

1. Select it under **Saved Profile**.
2. Press **Load Selected Profile**.
3. For a P25 system change, press **Stop**, load the profile, then press **Start scanning + audio** so OP25 regenerates and uses the selected system.

When scanning is running, the analog workers re-read their channel file between FFT sweeps. When scanning is stopped, an updated analog list becomes active the next time **Start scanning + audio** is pressed.

Delete a profile

Select it and press **Delete Selected**. Deleting a saved profile does not by itself erase the currently active runtime configuration.

Name a profile during upload

Enter **New Profile Name** before choosing or uploading the CSV. If the field is blank, N0JCG Scanner converts the CSV filename into the profile name.

13. Import and export analog CHIRP CSV files

Use the **Analog VHF / UHF** card on the Radio setup screen.

1. Press **Download Template**.
2. Edit the file in CHIRP, Excel, LibreOffice, or a text editor without changing the header names.
3. Enter the desired **New Profile Name**.
4. Choose the CSV file.
5. Press **Upload & Save Profile**.
6. Confirm the displayed VHF/UHF channel counts.

Required CHIRP columns are **Location**, **Name**, **Frequency**, and **Mode**. The downloaded template includes the full standard CHIRP column set.

Import behavior:

- 136–174 MHz channels are assigned to VHF / **analog_2m** / **<VHF_SERIAL>**.
- 400–520 MHz channels are assigned to UHF / **analog_70cm** / **<UHF_SERIAL>**.
- **FM** and **NFM** are accepted and normalized for the NFM scanner path.
- **Skip** values **S** or **L** import the row as disabled.
- **CHIRP Tone** by itself is transmit-only and does not enable receive gating.
- **TSQL**, **TSQL-R**, or a compatible cross mode uses **cToneFreq** as a receive CTCSS gate.
- A file containing only one band replaces only that band and preserves the other active band.

To back up a named analog profile, select it and press **Export Selected**.

14. Import and export P25 CSV files

Use the **P25 Systems & Talkgroups** card on the Radio setup screen.

1. Press **Download Template**.
2. Enter one row per control frequency, optional voice frequency, or talkgroup.
3. Enter a **New Profile Name**.
4. Choose the CSV and press **Upload & Save Profile**.
5. Press **Stop**, apply the different system profile, and then press **Start scanning + audio**. The coordinated controls restart P25, VHF, and UHF together.

Important columns:

Column	Use
RecordType	control, voice, or talkgroup
System	System name; required on every populated row
Site	Site name shown in configuration
FrequencyMHz	Required for control and voice rows
TGID	Required for talkgroup rows
Name	Talkgroup label
Enabled	true or false
Priority	Optional value from 0 through 100
NAC	Optional P25 network access code
Modulation	Modulation term shown by your frequency reference, such as C4FM or CQPSK
ControlDemod	Required. Control-channel modulation term shown by your reference. Enter C4FM, 4FSK, or CQPSK; N0JCG Scanner translates these to OP25's internal <code>fsk4</code> or <code>cqpsk</code> values.

Every imported system must have at least one enabled control-channel row. Every P25 profile must define `control_demod_type`; this value controls the control-channel demodulator used by OP25. Use the modulation term published by your system reference. C4FM and 4FSK are translated to OP25's `fsk4`, while CQPSK is translated to `cqpsk`. Do not rely on a legacy runtime marker or an environment override to select the control demodulator. Profiles may also define `preferred_control_channel_hz`. This frequency must be one of the listed control channels and is attempted first at startup. The remaining listed channels are alternates and are searched only if the preferred channel does not lock. If the field is omitted for an older profile, the first control-channel entry is treated as the preferred channel. Talkgroup IDs must be unique within the system and range from 0 through 65535. N0JCG Scanner does not decode encrypted traffic even if an encrypted talkgroup is present in a CSV.

To download a selected named P25 profile, press **Export Selected** in the P25 card.

15. Understand the audio arbitrator

Audio arrives on three loopback UDP ports:

Source	UDP port
P25	23456
VHF	23458
UHF	23459

The first source that produces enough valid frames becomes active. Other sources are rejected while that call owns the output. After approximately 1.5 seconds without frames, the arbitrator releases the source and can accept the next P25, VHF, or UHF call.

Each connected browser receives its own copy of the 8 kHz, mono, signed 16-bit PCM stream from port 8072. Browsers do not compete for audio frames. Each tab keeps only a small queue to limit delay; mute and volume operate in that browser, not in the radio workers or other tabs.

The arbitrator maintains a continuous 20 ms browser stream even through short source-packet gaps. A short three-frame server prebuffer and approximately 60 ms browser jitter buffer reduce clipped call beginnings while absorbing normal network and browser scheduling variation.

P25 audio is forwarded from the first valid decoded frame after each OP25 call boundary. The audio pool does not discard opening frames; its RMS validation, single-source ownership, and DRAIN/DROP boundary handling remain active.

Check audio status directly:

```
curl -fsS http://127.0.0.1:8072/api/audio/status \
| python3 -m json.tool
```

16. Routine maintenance and backups

Check service health

```
systemctl --no-pager --full status pi-p25-scanner.service
systemctl --no-pager --full status pi-p25-raw-audio-bridge.service
systemctl --no-pager --full status pi-p25-audio-pool.service
systemctl --no-pager --full status pi-scanner-vhf-worker.service
systemctl --no-pager --full status pi-scanner-uhf-worker.service
```

Interpret the results according to the dashboard state:

- Before **Start scanning + audio**, the backend, audio bridge, and audio pool should be active; the VHF and UHF workers should be inactive.
- While scanning, all five services should be active and `/api/status` should report P25 running, VHF active, and UHF active under `coordinated_scanners`.
- After **Stop**, the VHF and UHF workers should again be inactive while the three core services remain active.

Back up operator data

At minimum, preserve:

```
/home/pi/n0jcg-scanner/runtime/settings/
```

Example:

```
stamp="$(date -u +%Y%m%dT%H%M%SZ)"
mkdir -p "/home/pi/n0jcg-scanner-backups/$stamp"
cp -a /home/pi/n0jcg-scanner/runtime/settings \
"/home/pi/n0jcg-scanner-backups/$stamp/settings"
```

Named profiles, P25 settings, analog channel lists, skips, and blocks are all under the application runtime settings directory.

Restart after maintenance

First press **Stop** in the dashboard. If the dashboard is unavailable, stop the analog workers directly before maintenance:

```
sudo systemctl stop pi-scanner-vhf-worker.service
sudo systemctl stop pi-scanner-uhf-worker.service
```

Restart only the boot-enabled application infrastructure:

```
sudo systemctl restart pi-p25-raw-audio-bridge.service
sudo systemctl restart pi-p25-audio-pool.service
sudo systemctl restart pi-p25-scanner.service
```

Leave VHF and UHF stopped. Open the dashboard and press **Start scanning + audio** when reception should resume. Worker and backend counters may restart, except the persistent P25 **Voice Calls** total.

17. Troubleshooting

usb_claim_interface error -6

Another process already owns that receiver. This is expected if a scanner service is running. Do not reinstall drivers first.

1. Identify ownership:

```
ps -ef | grep -E 'rtl_tcp|rtl_fm|rx.py|multi_rx|readsb|dump978'
```

2. Press **Stop** so P25, VHF, and UHF release their receivers together. If the dashboard is unavailable, stop both analog worker services directly.
3. Run the bounded hardware test again.
4. Press **Start scanning + audio** when finished.

A N0JCG Scanner receiver is missing

- Check the powered hub and Pi power supply.
- Reseat one receiver at a time.
- Run `lsusb` and `rtl_test -t` with receiver-owning services stopped.
- Check for duplicate or blank serials with `rtl_eeeprom -d <temporary-index>`.
- Do not solve a missing device by changing persistent roles to current indexes.

The wrong receiver is scanning VHF or UHF

Check both the inventory registry and analog runtime status. VHF must be the configured VHF serial; UHF must use its configured UHF serial. Reapply the station role map and correct `analog_receivers.json`, then use **Stop** followed by **Start scanning + audio**. Do not enable either analog worker for boot.

Dashboard says Offline

```
systemctl is-active pi-p25-scanner.service
curl -fsS http://127.0.0.1:8070/api/status | python3 -m json.tool
journalctl -u pi-p25-scanner.service -n 100 --no-pager
```

No browser audio

- Tap **Start scanning + audio**; browser autoplay requires a user gesture.
- Confirm `/api/status` reports P25 `running`, VHF `active`, and UHF `active` under `coordinated_scanners`.
- Confirm the button says **Mute**, not **Unmute**, and raise the volume.
- Check whether **Active Source** shows P25, VHF, or UHF.
- Confirm port 8072 is reachable and the arbitrator service is active.
- Reload the page after restarting the audio service.

VHF or UHF never locks

- Confirm **Start scanning + audio** has been pressed and the applicable worker service is active.
- Confirm the frequency is enabled in the active analog profile.
- Confirm the correct band assignment and serial.
- Check antenna, feed line, filters, and receiver gain.
- Inspect status and recent worker logs:

```
curl -fsS http://127.0.0.1:8070/api/analog/status | python3 -m json.tool
journalctl -u pi-scanner-vhf-worker.service -n 100 --no-pager
journalctl -u pi-scanner-uhf-worker.service -n 100 --no-pager
```

A noisy analog channel repeatedly opens

Use **Skip 10 min** while evaluating it. Use **Block channel** if it should remain excluded. **Clear blocks** restores every skipped and blocked analog frequency.

P25 does not lock or follow calls

- Confirm the configured P25 control and voice serials are present.
- Confirm the selected system contains the correct current control channels.
- Check **Logs and details** for launch readiness and OP25 messages.
- Inspect `runtime/settings/op25_validated_rx_command.env` and the generated files under `runtime/op25/`.
- Confirm the private station command markers match its validated values:

```
P25_CONTROL_GAIN=<VALIDATED_CONTROL_GAIN>
P25_VOICE_GAIN=<VALIDATED_VOICE_GAIN>
P25_CONTROL_DEMOD_TYPE=<VALIDATED_CONTROL_DEMOD>
P25_VOICE_DEMOD_TYPE=<VALIDATED_VOICE_DEMOD>
P25_VOICE_SAMPLE_RATE=<VALIDATED_SAMPLE_RATE>
P25_VOICE_CENTER_HZ=<VALIDATED_CENTER_HZ>
```

- Use `tools/p25_terminal_diagnostic.py` to capture receiver frequency error and `tools/p25_terminal_plot_snapshot.py` for bounded spectrum and constellation evidence. Do not leave diagnostic plots enabled during normal operation.
- Run the bounded project probes before changing OP25 command arguments.

CSV upload fails

- Start from the downloadable template.
- Preserve exact header spelling.
- Save as UTF-8 CSV.
- Remove formulas, merged cells, and extra title rows.
- Confirm frequencies are in MHz, not Hz.
- For P25, include one enabled control row per system.

18. Technical reference

Network ports

Port	Protocol	Purpose
8070	TCP/HTTP	N0JCG Scanner web UI and backend API
8072	TCP/HTTP	Audio status and browser PCM/WAV stream
23456	UDP loopback	P25 audio into the arbitrator
23458	UDP loopback	VHF audio into the arbitrator
23459	UDP loopback	UHF audio into the arbitrator
23500–23509	UDP loopback	P25 multi-receiver audio pool inputs

Important runtime files

File	Purpose
<code>/home/pi/n0jcg-scanner/runtime/settings/p25_systems.json</code>	Active P25 system and talkgroups
<code>/home/pi/n0jcg-scanner/runtime/settings/receiver_roles.json</code>	Stable N0JCG Scanner receiver registry
<code>/home/pi/n0jcg-scanner/runtime/settings/configs/</code>	Named radio profiles

<code>/home/pi/n0jcg-scanner/runtime/settings/runtime_activity.json</code>	Persistent P25 Voice Calls total
<code>/home/pi/n0jcg-scanner/runtime/settings/analog_receivers.json</code>	Active VHF/UHF channels and worker settings
<code>/home/pi/n0jcg-scanner/runtime/settings/analog_controls.json</code>	Analog skips, blocks, and clear-lock requests
<code>/home/pi/n0jcg-scanner/runtime/status/analog_2m.json</code>	Current VHF worker status
<code>/home/pi/n0jcg-scanner/runtime/status/analog_70cm.json</code>	Current UHF worker status
<code>/home/pi/n0jcg-scanner/runtime/status/vhf_last_call.wav</code>	Most recent accepted VHF browser-audio capture
<code>/home/pi/n0jcg-scanner/runtime/status/uhf_last_call.wav</code>	Most recent accepted UHF browser-audio capture

Useful API endpoints

```
GET /api/status
GET /api/activity
GET /api/analog/status
GET /api/analog/channels
GET /api/analog/controls
GET /api/receivers/inventory
GET /api/config
GET /api/config/named
POST /api/scanner/start
POST /api/scanner/stop
```

`POST /api/scanner/start` is the coordinated Start action for P25, VHF, and UHF. `POST /api/scanner/stop` is the coordinated Stop action. The `GET /api/status` response includes `coordinated_scanners`, which reports the last known P25, VHF, and UHF state.

Registration and five-minute trial

The status bar shows **Registered**, **Unregistered**, or a live **Trial** countdown. An unregistered scanner operates for five minutes after scanning is started, then automatically stops P25, VHF, and UHF. To register, open **Menu** → **Registration**, enter the license S/N supplied by N0JCG and the purchaser email, then select **Activate license**. The scanner validates over HTTPS and keeps a signed offline lease. A registered scanner can remain offline for up to seven days after its last successful validation.

19. Acceptance checklist

Use this checklist after initial setup, a power cycle, or a major update:

- ☐ Pi boots without undervoltage or USB power warnings.
- ☐ Four unique N0JCG Scanner RTL-SDR serials are visible.

- [] P25 control and P25 voice match the private station role map.
- [] VHF and UHF match the private station role map.
- [] Receiver inventory reports no missing, duplicate, or unassigned serials.
- [] Immediately after boot, the backend, audio bridge, and audio pool are active, while P25, VHF, and UHF scanning are stopped.
- [] VHF and UHF worker services are not enabled for boot.
- [] ROC `<ROC_HOST>:8095/` shows an **Open Scanner** link to `http://<RADIO_HOST>:8070/`.
- [] ROC root `<ROC_HOST>:8095/` still loads the main dashboard.
- [] Radio Pi `<RADIO_HOST>:8070/api/status` returns the scanner state.
- [] Radio host `<RADIO_HOST>:8072/api/audio/status` reports `"ok": true`.
- [] `./tools/validate_split_runtime.sh` reports `FINAL=PASS` when the ROC dashboard is available; direct Pi operation remains valid without ROC.
- [] **Start scanning + audio** starts P25, VHF, and UHF and connects browser audio.
- [] While scanning, `/api/status` reports `P25 running`, `VHF active`, and `UHF active` under `coordinated_scanners`.
- [] VHF and UHF report healthy FFT-scanning states and nonzero channel counts.
- [] A known P25 call updates talkgroup information and Voice Calls.
- [] A known VHF transmission produces a VHF lock and complete audio.
- [] A known UHF transmission produces a UHF lock and complete audio.
- [] Skip, Block, Clear lock, and Clear blocks behave as expected.
- [] **Stop** stops P25, VHF, and UHF while leaving the dashboard online.
- [] A named profile can be saved, exported, loaded, and restored.

When every applicable item passes, the scanner is ready for normal operation.